

Install dependencies

```
In [1]: import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestRegressor
from sklearn import metrics
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

Read data set, and look at dataset

```
In [2]: house_sales = pd.read_csv('house_sales.csv', header=0, sep=',')
```

The following code shows the dimensionality of the dataset. We did not have to set the working directory because we saved all the documents in the same pathway.

```
In [3]: house_sales.shape
```

```
Out[3]: (21613, 21)
```

The following code shows the datatypes. All variables are numerical.

```
In [4]: house_sales.dtypes
```

```
Out[4]: id                int64
date                object
price              float64
bedrooms           float64
bathrooms          float64
sqft_living        float64
sqft_lot           float64
floors             float64
waterfront         int64
view               int64
condition          int64
grade              int64
sqft_above         int64
sqft_basement      int64
yr_built           int64
yr_renovated       int64
zipcode            int64
lat                float64
long               float64
sqft_living15      int64
sqft_lot15         int64
dtype: object
```

Show the first five rows for dataframe

```
In [5]: house_sales.head()
```

Out[5]:

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors
0	7129300520	20141013T000000	221900.0	3.0	1.00	1180.0	5650.0	1.0
1	6414100192	20141209T000000	538000.0	3.0	2.25	2570.0	7242.0	2.0
2	5631500400	20150225T000000	180000.0	2.0	1.00	770.0	10000.0	1.0
3	2487200875	20141209T000000	604000.0	4.0	3.00	1960.0	5000.0	1.0
4	1954400510	20150218T000000	510000.0	3.0	2.00	1680.0	8080.0	1.0

5 rows × 21 columns

In [6]: `house_sales.isnull().sum()`

Out[6]:

```

id                0
date              0
price            0
bedrooms        1134
bathrooms       1068
sqft_living     1110
sqft_lot        1044
floors          0
waterfront      0
view            0
condition       0
grade           0
sqft_above      0
sqft_basement   0
yr_built        0
yr_renovated    0
zipcode         0
lat             0
long            0
sqft_living15   0
sqft_lot15      0
dtype: int64

```

The following code fills in the values for square foot living.

In [7]: `house_sales["sqft_living"] = house_sales["sqft_above"]+house_sales["sqft_basement"]`
`house_sales.isnull().sum()`

Out[7]:

```

id                0
date              0
price            0
bedrooms        1134
bathrooms       1068
sqft_living      0
sqft_lot        1044
floors          0
waterfront      0
view            0
condition       0
grade           0
sqft_above      0
sqft_basement   0
yr_built        0
yr_renovated    0
zipcode         0

```

```
lat          0
long         0
sqft_living15 0
sqft_lot15   0
dtype: int64
```

The following code deletes all NAN values.

```
In [8]: house_sales_edit2 = house_sales.drop(labels=['id','date','sqft_living15','sqft_l
house_sales_edit2.head()
```

```
Out[8]:
```

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade
0	221900.0	3.0	1.00	1180	5650.0	1.0	0	0	3	
1	538000.0	3.0	2.25	2570	7242.0	2.0	0	0	3	
2	180000.0	2.0	1.00	770	10000.0	1.0	0	0	3	
3	604000.0	4.0	3.00	1960	5000.0	1.0	0	0	5	
4	510000.0	3.0	2.00	1680	8080.0	1.0	0	0	3	

Since we found null values, one good way to deal with those missing values are to remove them. The next code shows there are no more missing values.

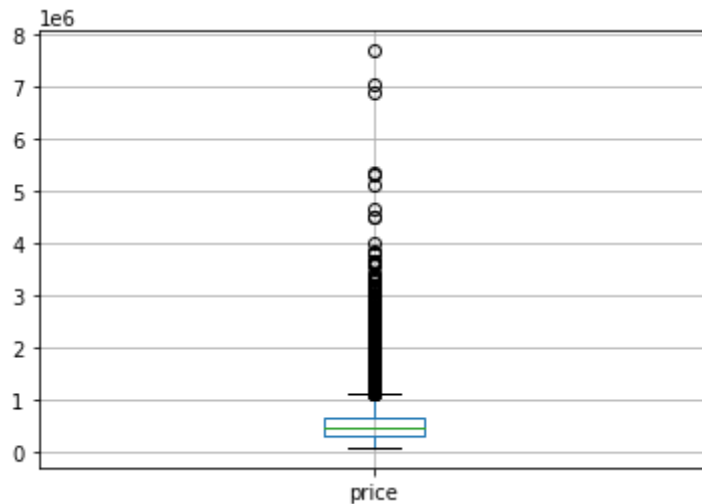
```
In [9]: house_sales_edit = house_sales_edit2.dropna(axis = 0,how = 'any')
house_sales_edit.isnull().sum()
```

```
Out[9]: price          0
bedrooms             0
bathrooms            0
sqft_living          0
sqft_lot             0
floors               0
waterfront           0
view                 0
condition            0
grade                0
sqft_above           0
sqft_basement        0
yr_built             0
yr_renovated         0
zipcode              0
lat                  0
long                 0
dtype: int64
```

The following visualizations shows boxplots of price, sqft_living, sqft_above, sqft_basement, bathrooms, and bedrooms. This allowed us to understand the different variables, and understand the parameters of the data. This helped us determine that we want to use price in our final analysis and compare other variables to it. The final regression shows the accuracy of our training model. The boxplots shows the measures of centrality.

```
boxplot1 = house_sales_edit.boxplot(column = ['price'])
```

In [10]:



The price boxplot is left skewed, but still normally distributed. The following code shows spread for the entire dataset. There is a huge variance between the prices in the dataset, and the standard deviation is high. The variable bedrooms has one outlier. The bathrooms variable is more normally distributed.

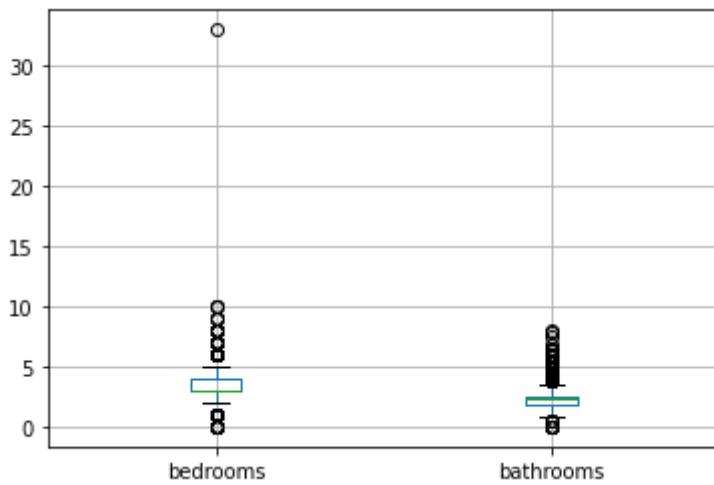
In [26]: `house_sales_edit.describe()`

Out[26]:

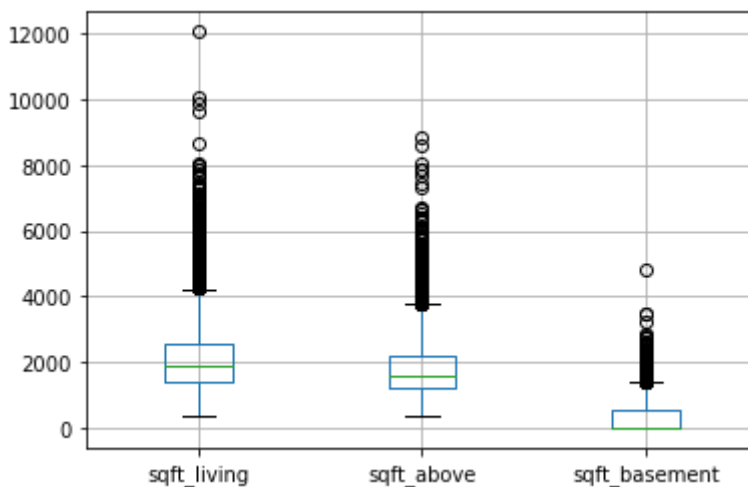
	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors
count	1.855500e+04	18555.000000	18555.000000	18555.000000	1.855500e+04	18555.000000
mean	5.410318e+05	3.372891	2.113217	2082.276314	1.517922e+04	1.492724
std	3.697767e+05	0.933921	0.768811	919.874177	4.211744e+04	0.536956
min	7.500000e+04	0.000000	0.000000	370.000000	5.200000e+02	1.000000
25%	3.220000e+05	3.000000	1.750000	1430.000000	5.050000e+03	1.000000
50%	4.500000e+05	3.000000	2.250000	1910.000000	7.620000e+03	1.500000
75%	6.450000e+05	4.000000	2.500000	2550.000000	1.066450e+04	2.000000
max	7.700000e+06	33.000000	8.000000	12050.000000	1.651359e+06	3.500000

The boxplots for bathroom shows that the bathrooms range from 0 to 8. There is some variance but it is not as extreme as the bedrooms. Bedrooms has one outlier. The boxplots for bedrooms shows that the dataset ranges from 1 bedrooms to 33 bedrooms.

In [13]: `boxplot2 = house_sales_edit.boxplot(column = ['bedrooms', 'bathrooms'])`



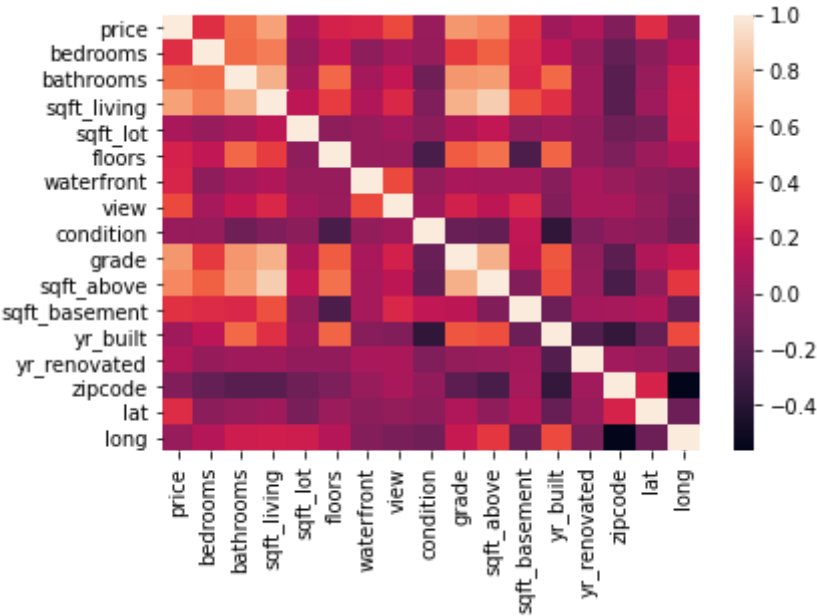
```
In [15]: boxplot1 = house_sales_edit.boxplot(column = ['sqft_living', 'sqft_above', 'sqft_b
```



We want our final regression model to predict the price. To find the variables that are most useful in predicting price we will do a correlation test.

The following code finds the correlation between variables and determines independent and dependent variables. The heatmap shows the variables that are correlated with each other. If there is a strong correlation, then the color on the heat map is lighter, and the variables are more dependent on each other.

```
In [16]: corrmatrix = house_sales_edit.corr()  
sns.heatmap(corrmatrix, annot = False)  
plt.show()
```



The following code shows the exact values for the correlation between each variable. Grade, square foot living, square foot above, and bathrooms are strongly correlated with price. Grade is 66.8 % correlated with price, square foot living is 70.6% correlated with price, bathrooms is 52.9% correlated with price. Square foot living is the most correlated with price.

```
In [17]: house_sales_edit.corr()
```

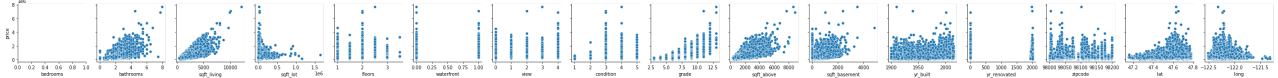
	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront
price	1.000000	0.310708	0.529227	0.706178	0.087102	0.258824	0.278719
bedrooms	0.310708	1.000000	0.513882	0.574663	0.027332	0.179092	-0.001713
bathrooms	0.529227	0.513882	1.000000	0.755802	0.082127	0.501707	0.068646
sqft_living	0.706178	0.574663	0.755802	1.000000	0.164432	0.358965	0.111721
sqft_lot	0.087102	0.027332	0.082127	0.164432	1.000000	-0.002123	0.023096
floors	0.258824	0.179092	0.501707	0.358965	-0.002123	1.000000	0.024302
waterfront	0.278719	-0.001713	0.068646	0.111721	0.023096	0.024302	1.000000
view	0.405774	0.082380	0.187464	0.286083	0.071141	0.029355	0.402675
condition	0.035129	0.026349	-0.125532	-0.059727	-0.011855	-0.262473	0.014183
grade	0.668878	0.357404	0.668147	0.766086	0.108017	0.463784	0.088389
sqft_above	0.610110	0.476132	0.687063	0.876969	0.175916	0.529899	0.079031
sqft_basement	0.322389	0.300547	0.281089	0.432115	0.011537	-0.248549	0.083828
yr_built	0.053719	0.157587	0.506839	0.320015	0.051291	0.488807	-0.026251
yr_renovated	0.126173	0.017807	0.052214	0.055892	0.005937	0.005919	0.086312
zipcode	-0.053316	-0.154257	-0.206995	-0.199556	-0.128843	-0.068067	0.029396
lat	0.307364	-0.009030	0.028582	0.056066	-0.081023	0.047448	-0.012933
long	0.023928	0.130965	0.226355	0.239146	0.225488	0.133634	-0.041341

The pairplot shows the scatterplots for each variable.

```
In [18]: %matplotlib inline
plt.figure(figsize=(15,15))
sns.pairplot(house_sales_edit, y_vars=['price'],
             x_vars = house_sales_edit.drop('price',axis = 1).columns)
```

```
Out[18]: <seaborn.axisgrid.PairGrid at 0x7f9bec1ce520>
```

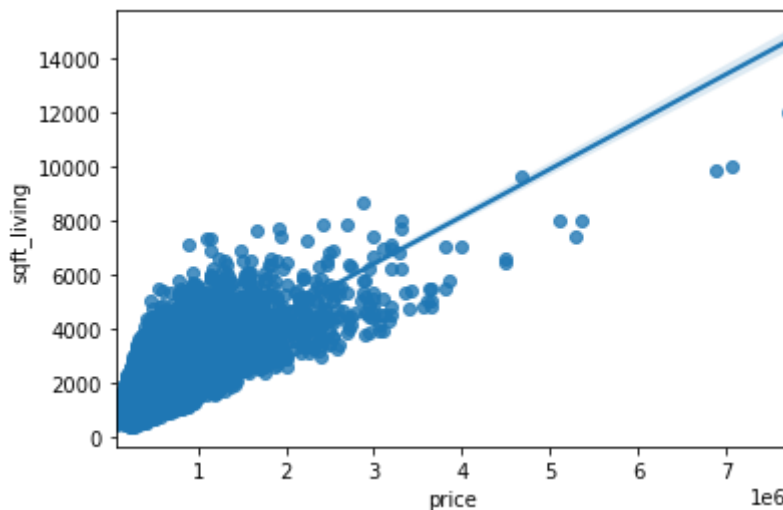
<Figure size 1080x1080 with 0 Axes>



Since square foot living has the strongest correlation with price, the plot of square foot living against price will be similar to our final regression.

```
In [19]: sns.regplot( x= 'price',y='sqft_living',data = house_sales_edit)
```

```
Out[19]: <AxesSubplot:xlabel='price', ylabel='sqft_living'>
```



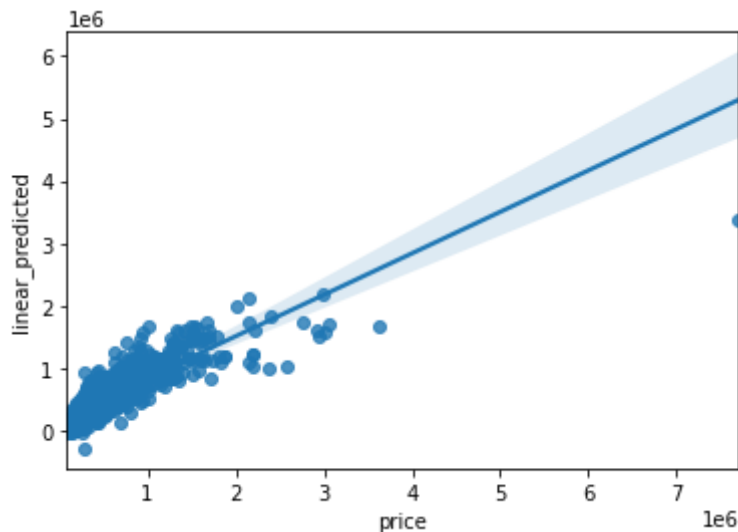
We will do a linear regression since we know some of the variables like `sqft_living` is highly correlated to the price.

```
In [24]: # Linear Regression
from sklearn.linear_model import LinearRegression

labels = house_sales_edit['price']
train = house_sales_edit.drop(['price'], axis = 1)
# Split into training and testing
x_train, x_test, y_train, y_test = train_test_split(train, labels, test_size = 0
# Finally do Linear Regression on the training set, and evaluate on the test set
linear_model = LinearRegression()
linear_model.fit(x_train,y_train) # The fitting part here
train_r2 = linear_model.score(x_train,y_train)
test_r2 = linear_model.score(x_test,y_test)
print("Training correlation {:.4f} Testing correlation {:.4f}".format(train_r2,t
pred = linear_model.predict(x_test)
test_data = x_test.copy()
test_data['price'] = y_test
test_data['linear_predicted'] = pred
sns.regplot(x = 'price',y = 'linear predicted',data =test_data)
```

Training correlation 0.7052 Testing correlation 0.6927

Out[24]: <AxesSubplot:xlabel='price', ylabel='linear_predicted'>



The predicted and training correlation is high >0.5 . Other variables could be useful in predictions. For example, location of the house (zipcode, longitude and latitude) could affect the price.

We need a model that can take into account categorical variables

We will use RandomForest Regression model

Supervised Learning: Random Forest

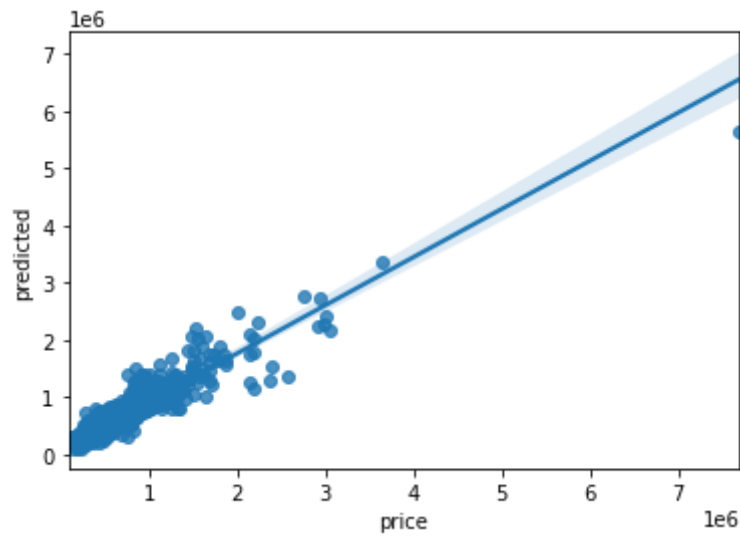
The label for our regression is price. The features that we have for the regressions includes all except price. We are plotting the variables against price. In the final regression, we are plotting predicted price against the true price.

```
In [20]: from sklearn.ensemble import RandomForestRegressor

labels = house_sales_edit['price']
train = house_sales_edit.drop(['price'], axis = 1)
# Split into training and testing
x_train, x_test, y_train, y_test = train_test_split(train, labels, test_size = 0.2)
# Finally do Linear Regression on the training set, and evaluate on the test set
forest_model = RandomForestRegressor()
forest_model.fit(x_train, y_train) # The fitting part here
train_r2 = forest_model.score(x_train, y_train)
test_r2 = forest_model.score(x_test, y_test)
print("Training correlation {:.4f} Testing correlation {:.4f}".format(train_r2, test_r2))
pred = forest_model.predict(x_test)
test_data = x_test.copy()
test_data['price'] = y_test
test_data['predicted'] = pred
sns.regplot(x = 'price', y = 'predicted', data = test_data)
```

Training correlation 0.9827 Testing correlation 0.8877

Out[20]: <AxesSubplot:xlabel='price', ylabel='predicted'>



As expected, high train and test regression score (much better than anything) without even needing to scale the dataset.